

Of course! I understand the pressure of a tough lab exam. I've analyzed your provided assignments and labs to create a targeted revision sheet for you. While I can't generate a PDF, I've formatted this as a comprehensive, easy-to-read guide covering the regular expressions and lex strategies you'll need, from the basics to the advanced patterns your professor might use.

Here is your revision sheet for the CSL302 Lexical Analysis lab exam.

CSL302: Lexical Analysis & Regex Revision Sheet

This guide is tailored specifically to the token patterns found in your CSL302 assignments and labs. Focus on understanding the *logic* behind the patterns, not just memorizing them.

1. Regex Fundamentals Quick Reference

First, a quick refresher on the building blocks.

Symbol/ Construct	Meaning	Example
.	Any character except newline	a.c matches "abc", "a_c", "a5c"
[abc]	Any one character from the set (a, b, or c)	[aeiou] matches any vowel
[a-z]	Any one character in the range 'a' to 'z'	[a-zA-Z0-9] matches any alphanumeric char
[^abc]	Any character NOT in the set	[^0-9] matches any non-digit
*	Zero or more repetitions	a* matches "", "a", "aa", "aaa"
+	One or more repetitions	a+ matches "a", "aa", but not ""
?	Zero or one repetition (optional)	colou?r matches "color" and "colour"
{n}	Exactly n repetitions	[0-9]{3} matches "123", "987"
{n,m}	Between n and m repetitions	[0-9]{2,4} matches "12", "123", "1234"
	Alternation (OR)	cat dog matches "cat" or "dog"

Symbol/ Construct	Meaning	Example
()	Grouping	(ab)+ matches "ab", "abab", "ababab"
^	Start of a line	^A matches an 'A' only at the start of a line
\$	End of a line	end\$ matches 'end' only at the end of a line
\	Escape character	\. matches a literal dot .

2. Patterns from Your Course Materials (Easy to Hard)

Here are the specific regular expressions derived directly from your assignments, progressing in complexity.

Level 1: Basic Tokens & Literals

These are straightforward patterns for common data types.

- **Decimal Integer:** A simple sequence of digits¹.

Code snippet

```
[0-9]+
```

- **Whitespace:** Spaces, tabs, and newlines that should be ignored². The action in lex would be {} (do nothing).

Code snippet

```
[ \t\n]+
```

- **Single-Line Comment (Assignment 1 style):** Starts with // and goes to the end of the line³.

Code snippet

```
"//".*
```

- **Simple Operators:** These are single or multi-character operators.

Important lex tip: Always list longer patterns before shorter ones (e.g., == before =) to ensure the correct token is matched⁴.

Code snippet

```
"=="    { return T_EQ; }
"="     { return T_ASSIGN; }
```

```

"<="    { return T_LE; }
"<"     { return T_LT; }
"&&"    { return T_AND; }
"|"     { return T_OR; }
"!="    { return T_NEQ; }
"!"     { return T_NOT; }
"+"     { /* return token */ }
/* ... and so on for others ... */

```

Level 2: Structured & Constrained Tokens

These tokens have specific rules about their format, starting characters, or length.

- **Identifier (Assignment 1):** Starts with a letter, followed by letters, digits, or underscores. Maximum length is 31 characters⁵⁵⁵⁵.

Code snippet

```
[a-zA-Z][a-zA-Z0-9_]{0,30}
```

- **Logic:** [a-zA-Z] for the first character, followed by {0,30} of the allowed characters to make a total maximum of 31.
- **In your lex file,** you'd need an action to check the length: { if (yyleng > 31) { /* handle error */ } else { return IDENTIFIER; } }
- **Hexadecimal Integer (Assignment 1):** Starts with 0x or 0X, followed by hex digits⁶.

Code snippet

```
0[xX][0-9a-fA-F]+
```

- **Phone Number (Lab 2):** Format (123) 456-89117.

Code snippet

```
\([0-9]{3}\)[ ]?[0-9]{3}-[0-9]{4}
```

- **Logic:** \ (and \) are used to match literal parentheses.
- **Email Address (Lab 2):** username@Domain with specific rules⁸.

Code snippet

```
[a-zA-Z][a-zA-Z0-9]*@[a-z]+\.[a-z]+\.[a-z]+
```

- **Breakdown:** [a-zA-Z][a-zA-Z0-9]* is the username, followed by @, then three dot-separated lowercase strings for the domain.

Level 3: Complex & Tricky Tokens 🔥

These are the most complex patterns and are very likely to appear in a tough exam.

- **Double / Floating Point (Assignment 1):** A sequence of digits, a required decimal point, and an optional exponent. Remember,

12. is valid but 12 is not9.

Code snippet

```
[0-9]+\.[0-9]*([eE][+-]?[0-9]+)?
```

- **Breakdown:**

- `[0-9]+\.` : One or more digits followed by a literal dot. This enforces the rule that the decimal point is required.
- `[0-9]*` : Zero or more digits after the decimal point.
- `([eE][+-]?[0-9]+)?` : An optional (?) group for the exponent. It contains e or E, an optional sign `[+-]?`, and one or more digits.

- **String Literal (Assignment 1):** Enclosed in double quotes, containing anything *except* a newline or another double quote10.

Code snippet

```
\"[^"\\n]*\"
```

- **Logic:** It starts and ends with ". The middle part `[^"\\n]*` means "match any character that is NOT a double quote or a newline, zero or more times".
- **Multi-line Comment (Assignment 1 Style):** Starts with `/*` and ends with the *first subsequent* `*/`. They do not nest11. This is a classic tricky pattern. A simple `\\*.*\\*/` is **wrong** because it's "greedy" and will match from the first `/*` to the very last `*/` in the file.

- **The Correct Way (using lex Start Conditions):** This is the most robust method and shows a deep understanding.

Code snippet

```
%x COMMENT
%%
<INITIAL>"/*"    BEGIN(COMMENT);
<COMMENT>"*/"    BEGIN(INITIAL);
<COMMENT>(.|\\n) /* Do nothing, just consume characters */
```

- **The "One-Liner" Regex (Advanced):** This pattern avoids being greedy.

Code snippet

```
\\*([\\^*]|\\*+[\\^*/])\\*+\\/
```

- **How it works:** It matches `/*`, then repeatedly matches either a character that isn't a star (`[\\^*]`) or a star that isn't followed by a slash (`\\*+[\\^*/]`), and finally ends with `*/`.

- **File Path (Lab 4):** A path with directories, .., and specific file extensions12121212.

Code snippet

```
(V([a-zA-Z0-9]+|\.\.))+(\.[a-z]+)?
```

- **Breakdown:**

- `V`: A literal slash.
- `([a-zA-Z0-9]+|\.\.)`: A directory name OR ...
- `(V([a-zA-Z0-9]+|\.\.))+`: A full path segment, repeated one or more times.
- `(\.[a-z]+)?`: An optional file extension. You could make this `\.(txt|dat)` to be more specific to your lab.

3. Key lex / flex Exam Strategies

1. **Order Matters:** lex matches rules in the order they appear. Always place more specific rules (like keywords) before more general rules (like identifiers).

Code snippet

```
"if"      { return T_IF; }
"else"    { return T_ELSE; }
[a-zA-Z]+ { return T_IDENTIFIER; } /* General rule comes last */
```

2. **Use Start Conditions:** For any token that has a distinct start and end delimiter and can contain almost anything in between (like multi-line comments or complex strings), start conditions (`%x STATE_NAME`) are the best and safest solution.
3. **Know yytext and yyleng:** yytext is a char* pointing to the matched string. yyleng is its length. You'll need this for checking identifier length limits or converting matched strings to numbers (atoi, atof).
4. **Handling "Unmatched" Characters:** Have a final "catch-all" rule at the bottom of your rules section to handle any characters that don't match any other token. This prevents your lexer from getting stuck.

Code snippet

```
. { printf("Error: Unexpected character '%s'\n", yytext); }
```

Good luck with your exam! Review these patterns and strategies, and you'll be well-prepared.

Of course. Based on your request, I've searched for other common but complex regular expression patterns that are frequently used in compiler construction and could appear on a difficult exam.

Here is an additional section for your revision sheet with more advanced patterns that go beyond your specific course materials but cover classic lexical analysis challenges.

4. Additional Advanced & Internet-Sourced Patterns

Your professor might test you on well-known lexical analysis problems to gauge your deeper understanding. Here are some common ones.

- **String Literals with Escape Sequences**

Real programming languages allow escaping characters within a string, like \" to represent a quote or \n for a newline. Your assignment's string definition was simpler¹. A regex to handle this is more complex.

Regex:

Code snippet

$$\backslash"(\backslash\backslash.[^"\backslash\backslash])*\backslash"$$

Breakdown:

- \": Matches the opening and closing double-quote.
- (...)*: The content of the string, which can be zero or more of the following:
 - \\. Matches a backslash \ followed by **any single character**. This correctly consumes escape sequences like \", \\, \t, \n, etc., as a single unit.
 - |: An OR condition.
 - [^"\\]: Matches **any single character that is NOT** a double-quote or a backslash.
- **Nested Comments**

Your assignment specifies that multi-line comments do not nest². A classic "hard" problem is to write a lexer that

can handle nested comments (e.g., `/* comment /* nested */ */`).

This cannot be solved reliably with a standard flex regular expression alone. Regular expressions are equivalent to finite automata, which cannot count. To handle nesting, you need a counter. This is a trick question if you're only allowed to use a single regex.

The lex/flex Solution (using C code): This is the correct way to solve it and demonstrates advanced knowledge.

Code snippet

```
%x C_COMMENT
int comment_depth = 0;
%%

<INITIAL>"/*"    { BEGIN(C_COMMENT); comment_depth = 1; }

<C_COMMENT>"/*"  { comment_depth++; }
<C_COMMENT>"*/"  {
    comment_depth--;
    if (comment_depth == 0) {
        BEGIN(INITIAL);
    }
}

<C_COMMENT>(.|\n) { /* Consume any character inside the comment */ }
```

This solution uses a start condition C_COMMENT and an integer comment_depth to track the nesting level.

- **Character Literals**

Languages like C and Java have character literals, which are distinct from strings. They are enclosed in single quotes and can also contain escape sequences.

Regex:

Code snippet

```
'(\\.|[^\"])'
```

Breakdown: This logic is very similar to the string literal with escapes, but it uses single quotes (') and expects exactly one character inside (notice there is no * quantifier). It matches either an escaped character (\\.) or a single character that is not a single-quote or backslash ([^\"]).

- **More Flexible Floating-Point Numbers**

Your assignment had very specific rules for doubles3333. A more general (and common) definition allows for numbers starting with a decimal point (e.g.,

.123) or numbers without a decimal point if they have an exponent (e.g., 1e10).

Regex:

Code snippet

```
([0-9]+\.[0-9]*|\.[0-9]+|[0-9]+)([eE][+-]?[0-9]+)?
```

Breakdown: This pattern is powerful because of the OR conditions for the main number part (the mantissa).

- `([0-9]+\.[0-9]*|\.[0-9]+|[0-9]+)`: This group matches the mantissa in one of three ways:
 - `[0-9]+\.[0-9]*`: The standard form, like 12.34 or 12..
 - `\.[0-9]+`: A number starting with a decimal point, like .123.
 - `[0-9]+`: An integer, which is needed for scientific notation like 5e7.
- `([eE][+-]?[0-9]+)?`: This is the same optional exponent part from the previous sheet.
- **Crucially**, for this to work correctly in lex, you'd place the rule for regular integers `([0-9]+)` *after* this rule, so that 123 is only matched as an integer if it's not part of a float like 123.45 or 123e5.

By studying both the patterns from your course and these common advanced cases, you will be very well-prepared for whatever your professor includes in the exam.

Good luck!